[NOME DA INSTITUICAO]
[NOME DO PROGRAMA]
Programa Institucional de Bolsas de Iniciacao Cientifica
(PIBIC/CNPq)

RELATORIO PARCIAL DE PESQUISA

PVM2263-2025
FLORA: Friendly Local Open Resource for Analysis

Bolsista: Joao Pedro Pereira Santiago
Orientador: Prof. Dr. Diego Lisboa Rios
Periodo de avaliacao: Agosto/2025 a Marco/2026

[Cidade – Estado]
Abril de 2026

Abstract

Este relatorio apresenta o estado parcial do projeto FLORA (*Friendly Local Open Resource for Analysis*), uma biblioteca cientifica em Python desenvolvida para analise ponta a ponta de dados de microbioma baseados em amplicons 16S rRNA. No periodo avaliado, o projeto deixou de ser apenas uma proposta de interface local e foi consolidado como uma base de software modular, com responsabilidades separadas em configuracao, ingestao, persistencia analitica em DuckDB, engenharia de atributos com Polars, diversidade alfa e beta, aprendizado de maquina, visualizacao, relatorios HTML e interface web local. A arquitetura implementada no repositorio organiza o fluxo entre fontes externas de dados publicos, armazenamento local em arquivos TSV/BIOM/Parquet, banco DuckDB e modulos de inferencia estatistica e preditiva, o que reduz acoplamento entre etapas e melhora reprodutibilidade. O projeto ja oferece download de bases publicas por conectores para MGnify, EMP e NCBI SRA, ingestao padronizada de `metadata.tsv`, `asv_table.tsv`, `taxonomy.tsv` e BIOM, bem como geracao de matrizes normalizadas por *raw*, TSS e CLR para modelagem com *scikit-learn* e XGBoost. Do ponto de vista de engenharia, o sistema encontra-se pronto para empacotamento como distribuicao Python sob o nome `flora-bio`, com *build backend* baseado em Hatchling, comando de entrada `flora` e organizacao compatível com publicacao futura em PyPI e integracao com documentacao tecnica. Em avaliacao experimental com subconjunto do Earth Microbiome Project, a pipeline processou 89 amostras, preservou 1.183 atributos apos filtragem de prevalencia e alcancou acuracia de 0,8636, F1-macro de 0,6381 e ROC-AUC de 0,9510 em tarefa de classificacao ambiental. Esses resultados indicam que o FLORA ja ultrapassou a fase de prototipo e se aproxima de um produto cientifico publicavel, reutilizavel e extensivel.

Palavras-chave: microbioma, 16S rRNA, DuckDB, bioinformatica, machine learning, software cientifico.

Contents

1. Introducao

A analise de microbiomas baseada em sequenciamento de amplicons 16S rRNA continua sendo uma estrategia central para caracterizacao de comunidades microbianas em ambientes naturais, hospedeiros e sistemas experimentais. A literatura consolidou ferramentas de alta relevancia para o processamento primario desses dados, como QIIME 2 [?] e DADA2 [?]. Entretanto, a etapa posterior ao processamento biologico ainda costuma ser fragmentada: o pesquisador precisa combinar manualmente arquivos de abundancia, tabelas taxonomicas, metadados, rotinas de normalizacao, analise de diversidade, algoritmos de aprendizado de maquina e ferramentas de visualizacao.

Esse acoplamento manual cria quatro dificuldades recorrentes. Primeiro, aumenta o custo operacional de cada novo experimento. Segundo, dificulta a reproducao analitica por diferentes membros de um grupo de pesquisa. Terceiro, enfraquece o rastreamento do que foi feito em cada etapa. Quarto, torna mais dificil transformar um pipeline experimental em biblioteca cientifica reutilizavel.

O projeto FLORA foi concebido para enfrentar esse problema por meio de uma arquitetura local, aberta e modular. Em vez de tratar a analise como uma sucessao de scripts independentes, o FLORA organiza o fluxo em camadas coerentes: conectores para dados externos, ingestao validada, armazenamento analitico embutido, engenharia de atributos, modelagem estatistica, visualizacao e geracao automatizada de relatorios. O foco do projeto no periodo atual deixou de ser apenas uma interface amigavel e passou a ser a consolidacao da biblioteca como infraestrutura cientifica.

Outro ponto central do trabalho e a natureza composicional dos dados de abundancia microbiana. Matrizes de ASVs representam distribuicoes relativas, nao contagens absolutas independentes, de forma que o uso direto de abundancias brutas pode introduzir correlacoes espurias. A literatura recomenda o uso de transformacoes do tipo log-ratio, especialmente CLR, para reduzir esse problema [?]. O FLORA incorpora esse principio na propria camada analitica, permitindo gerar matrizes normalizadas de forma padronizada e reprodutivel.

2. Objetivos

2.1. Objetivo Geral

Desenvolver uma biblioteca e plataforma local em Python para analise ponta a ponta de dados de microbioma por amplicons 16S rRNA, integrando aquisicao de dados, armazenamento analitico, transformacoes composicionais, aprendizado de maquina, visualizacao e geracao automatizada de relatorios.

2.2. Objetivos Especificos

- consolidar o FLORA como biblioteca modular, e nao apenas como interface de demonstracao;
- integrar o fluxo entre dados externos publicos, armazenamento local e banco analitico embutido;
- estruturar a persistencia em DuckDB com esquema reprodutivel para amostras, ASVs, taxonomia, diversidade e log de pipeline;
- implementar engenharia de atributos adequada para dados composicionais, com suporte a TSS e CLR;
- disponibilizar componentes de classificacao, regressao, clustering e interpretabilidade;
- produzir documentacao tecnica suficiente para publicacao futura da biblioteca e uso por outros pesquisadores.

3. Metodologia

3.1. Visao geral da arquitetura

O desenvolvimento atual do FLORA foi organizado em pacotes Python com responsabilidades bem definidas, refletindo a estrutura do repositorio `src/flora/`. A biblioteca ja contem modulos para `config`, `core`, `db`, `io`, `diversity`, `feature_engineering`, `ml`, `reports`, `viz`, `pipelines` e `ui`. Essa divisao nao e meramente cosmetica: ela reduz o acoplamento entre regras de negocio, persistencia, conectores externos e camada de apresentacao.

Do ponto de vista operacional, o objeto `FLORAPipeline` atua como orquestrador de alto nivel, enquanto `FloraDB` concentra o acesso ao DuckDB e os conectores em `flora.io.downloaders` concentram a comunicacao com bases externas. A Figura em Mermaid abaixo resume esse desenho.

```
flowchart TD
    A[Dados externos\nMGnify / EMP / NCBI SRA] --> B[Downloaders em flora.io.
    downloaders]
    B --> C[Arquivos locais\nFASTQ / BIOM / TSV / manifest.tsv / metadata.tsv]
    C --> D[FLORAPipeline]
    D --> E[Ingestao em flora.db.ingestion]
    E --> F[(DuckDB local\nflora.duckdb)]
    F --> G[Views analiticas\nv_asv_tss / v_asv_clr / v_taxonomy_full /
    v_diversity_wide]
```

```
    G --> H[Feature engineering\nPolars + SQL]
    H --> I[Diversidade alfa e beta]
    H --> J[ML\nClassificacao / Regressao / Clustering]
    J --> K[Interpretabilidade\nFeature importance / SHAP]
    I --> L[Visualizacoes]
    J --> L
    K --> L
    L --> M[FLORAReport HTML]
    F --> N[UI local\nCLI + servidor HTTP + SPA Angular]
```

Listing 1: Diagrama Mermaid de alto nivel da arquitetura do FLORA.

O diagrama **??** evidencia uma decisao de engenharia importante: o banco local e a camada central de articulacao entre ingestao, transformacao, visualizacao e aprendizado de maquina. Em vez de cada etapa escrever arquivos temporarios em formatos distintos, o projeto concentra o estado analitico em um modelo persistente e consultavel.

3.2. Diagrama de classes do nucleo do sistema

O nucleo do software pode ser descrito por meio de poucas classes de alta responsabilidade. O principal ponto de entrada programatico e `FLORAPipeline`, que inicializa configuracao, workdir, logging e conexao com o banco. A classe `FloraDB` encapsula a conexao DuckDB e expande essa conexao com metodos de analise. Os objetos de ML, relatarios e interface local funcionam como consumidores dessa base.

```
classDiagram
    class FloraConfig {
        +database
        +pipeline
        +ml
        +logging
        +storage
    }

    class FLORAPipeline {
        +config : FloraConfig
        +workdir : Path
        +db : FloraDB
        +ingest_metadata(path)
        +ingest_asv_table(path, wide_format)
        +ingest_taxonomy(path)
```

```
    +compute_diversity(sampling_depth, metrics)
    +get_feature_matrix(normalize, min_prevalence)
    +close()
}

class FloraDB {
    +connect(path, threads, memory_limit)
    +initialize_schema()
    +create_views()
    +load_dataframe(table, df, if_exists)
    +load_parquet(table, parquet_path)
    +pivot_asv(normalize, min_prevalence)
    +aggregate_by_taxon(level, group_by, metric)
    +slice(train_filter, test_filter, features, target_column)
    +table_info()
    +query(sql)
    +close()
}

class QueryResult {
    +to_polars()
    +to_pandas()
    +to_arrow()
    +to_parquet(path)
}

class MicrobiomeClassifier {
    +fit(train_df, test_df, cv_folds, run_name)
}

class MicrobiomeRegressor {
    +fit(train_df, test_df, cv_folds)
}

class MicrobiomeClusterer {
    +fit(df)
}

class SHAPAnalyzer {
    +explain(df)
}
```

```
class FLORAReport {
    +add_metrics(title, metrics)
    +add_table(title, df)
    +add_plot(title, fig)
    +save(path)
}

class FLORAState {
    +config
    +workdir
    +pipeline
}

class MGnifyDownloader {
    +fetch(study_accession, output_dir, max_samples)
    +fetch_analyses(study_accession, output_dir, max_analyses)
}

class NCBISRADownloader {
    +fetch(accessions, output_dir)
}

class EMPDownloader {
    +fetch(output_dir, max_samples)
}

FLORAPipeline --> FloraConfig
FLORAPipeline --> FloraDB
FloraDB --> QueryResult
FLORAPipeline --> MicrobiomeClassifier
FLORAPipeline --> MicrobiomeRegressor
FLORAPipeline --> MicrobiomeClusterer
MicrobiomeClassifier --> SHAPAnalyzer
FLORAPipeline --> FLORAReport
FLORAState --> FLORAPipeline
FLORAPipeline --> MGnifyDownloader
FLORAPipeline --> NCBISRADownloader
FLORAPipeline --> EMPDownloader
```

Listing 2: Diagrama Mermaid de classes do nucleo do FLORA.

Esse diagrama esta diretamente alinhado a classes reais do repositorio, encontradas em arquivos como `src/flora/pipelines/main_pipeline.py`, `src/flora/db/connection.py`, `src/flora/ml/classification/classifier.py`, `src/flora/reports/html_report.py` e `src/flora/io/downloaders.py`. Isso e importante porque o relatorio passa a documentar a implementacao existente, e nao apenas uma arquitetura idealizada.

3.3. Conexao com dados externos e armazenamento local

Uma contribuicao importante do projeto no periodo foi a consolidacao da camada de conectores para dados publicos. O arquivo `src/flora/io/downloaders.py` implementa tres pontos de entrada:

- **MGnifyDownloader**: acessa a API REST publica do EMBL-EBI, busca amostras, *runs*, arquivos FASTQ e tambem pode baixar analises precomputadas;
- **NCBISRADownloader**: integra o ecossistema do NCBI SRA para baixar experimentos por accession;
- **EMPDownloader**: organiza o acesso a dados do Earth Microbiome Project.

Esses conectores nao entregam dados diretamente ao modelo. Eles primeiro constroem um estado local reprodutivel em disco, tipicamente sob `data/raw/` ou outro diretório configurado pelo usuario. O FLORA escreve `manifest.tsv`, `metadata.tsv`, arquivos `fastq/`, tabelas `asv_table.tsv` e `taxonomy.tsv` quando aplicavel. Depois disso, a ingestao converte os dados para DataFrames e os persiste no banco analitico.

```
flowchart LR
    subgraph Externo
        A[MGnify REST API]
        B[NCBI SRA]
        C[EMP portal / Qiita]
    end

    subgraph Conectores
        D[MGnifyDownloader]
        E[NCBISRADownloader]
        F[EMPDownloader]
    end

    subgraph DiscoLocal
        G[data/raw/fastq]
        H[metadata.tsv]
```

```
        I[manifest.tsv]
        J[asv_table.tsv]
        K[taxonomy.tsv]
        L[cache parquet/]
    end

    subgraph Ingestao
        M[ingest_metadata]
        N[ingest_tsv_asv / ingest_biom]
        O[ingest_taxonomy]
    end

    A --> D
    B --> E
    C --> F
    D --> G
    D --> H
    D --> I
    D --> J
    D --> K
    E --> G
    E --> I
    F --> H
    F --> J
    F --> K
    H --> M
    J --> N
    K --> O
    N --> L
    M --> P[(DuckDB)]
    N --> P
    O --> P
```

Listing 3: Diagrama Mermaid do fluxo entre fontes externas, arquivos locais e ingestao.

Esse fluxo e relevante para o objetivo da pesquisa porque o FLORA nao depende apenas de dados sinteticos ou de um formato unico de entrada. Ele permite baixar, organizar, validar e ingerir dados reais, aproximando a biblioteca de um uso efetivo por pesquisadores de microbioma.

3.4. Banco local DuckDB

3.4.1. Motivacao para uso do DuckDB

O DuckDB [**?**] foi adotado como motor analitico central do FLORA porque oferece operacao *in-process*, boa performance para consultas colunares, suporte a SQL analitico e interoperabilidade com Arrow e Parquet. Na pratica, isso significa que o pipeline pode manipular grandes matrizes de abundancia e combinacoes entre taxonomia e metadados sem depender de um servidor de banco externo. Para o contexto de um projeto PIBIC, isso e estrategico: o sistema continua local, leve e reproduzivel.

No codigo, a classe `FloraDB` configura a conexao, ajusta `threads` e `memory_limit`, executa DDL, carrega DataFrames Polars e oferece metodos de consulta e transformacao. O banco pode ser efemero em `:memory:` para testes ou persistente em arquivo, tipicamente `results/flora.duckdb`, para execucao real.

3.4.2. Esquema relacional implementado

O esquema do banco esta definido em `src/flora/db/schema.py`. O desenho segue uma estrutura proxima a *star schema*, com tabelas centrais de fatos e dimensoes analiticas. A Tabela **??** resume o papel de cada tabela.

Table 1: Documentacao das tabelas locais do DuckDB utilizadas no FLORA.

Tabela	Tipo	Descricao e campos principais
`samples`	dimensao	Uma linha por amostra biologica. Guarda `sample_id`, `biome`, `location`, `latitude`, `longitude` e `sequencing_depth`.
`sample_metadata`	extensao de metadados	Armazena pares `(sample_id, key, value)` para colunas extras nao previstas no nucleo da tabela `samples`. Permite flexibilidade sem perder normalizacao.
`asv`	fato principal	Uma linha por observacao `(sample_id, feature_id)` com campo `abundance`. E a tabela mais importante para transformacoes, pivots e agregacoes taxonomicas.
`taxonomy`	dimensao taxonomica	Mapeia `feature_id` para `kingdom`, `phylum`, `class`, `order`, `family`, `genus`, `species` e `confidence`.

Tabela	Tipo	Descricao e campos principais
`diversity_alpha`	resultado derivado	Guarda metricas por amostra em formato longo: `sample_id`, `metric`, `value`, `sampling_depth`.
`diversity_beta`	resultado derivado	Guarda distancias pareadas entre amostras com chaves `sample_a`, `sample_b`, `metric` e `distance`.
`dim_reduction`	resultado derivado	Estrutura prevista para armazenar componentes de reducao de dimensionalidade por amostra, metodo e componente.
`pipeline_log`	auditoria	Registra `run_id`, etapa, status, tempos de inicio e fim, parametros e mensagens, permitindo rastreabilidade do pipeline.

Alem das tabelas, o FLORA cria *views* padrao em `FloraDB.create_views()`. Entre elas estao `v_asv_tss`, `v_asv_clr`, `v_taxonomy_full` e `v_diversity_wide`. Essas *views* funcionam como uma camada analitica intermediaria: deixam prontas consultas recorrentes sem duplicar dados fisicamente.

```
classDiagram
    class samples {
        sample_id PK
        biome
        location
        latitude
        longitude
        sequencing_depth
    }

    class sample_metadata {
        sample_id FK
        key PK
        value
    }

    class asv {
        sample_id FK
        feature_id PK
        abundance
```

```
}

class taxonomy {
    feature_id PK
    kingdom
    phylum
    class
    order
    family
    genus
    species
    confidence
}

class diversity_alpha {
    sample_id FK
    metric PK
    value
    sampling_depth
}

class diversity_beta {
    sample_a
    sample_b
    metric
    distance
}

class dim_reduction {
    sample_id FK
    method
    component
    value
}

class pipeline_log {
    run_id
    step
    status
    started_at
    finished_at
```

```
        params
        message
    }

    samples "1" --> "*" sample_metadata : possui
    samples "1" --> "*" asv : observa
    taxonomy "1" --> "*" asv : anota
    samples "1" --> "*" diversity_alpha : resume
    samples "1" --> "*" dim_reduction : projeta
```

Listing 4: Diagrama Mermaid do esquema logico do banco DuckDB do FLORA.

3.4.3. Como o DuckDB se conecta com o resto do sistema

No FLORA, o banco nao e apenas um repositório passivo. Ele fornece operacoes analiticas que simplificam diretamente as etapas posteriores do pipeline:

- `pivot_asv()` gera matrizes amostra × feature com suporte nativo a *raw*, TSS e CLR;
- `aggregate_by_taxon()` junta `asv`, `taxonomy` e `samples` para sumarizacoes biologicas;
- `slice()` produz subconjuntos treino/teste com filtros SQL;
- `query()` e `QueryResult` permitem exportacao para Polars, Pandas, Arrow e Parquet.

Em outras palavras, o banco local e tambem uma API analitica. Isso elimina a necessidade de reimplementar a mesma logica em diferentes scripts e torna a arquitetura muito mais consistente.

3.5. Engenharia de atributos, diversidade e aprendizado de maquina

As transformacoes tabulares sao implementadas principalmente com Polars, enquanto as operacoes relacionais e de agregacao ficam no DuckDB. Essa combinacao se mostrou adequada para dados de microbioma porque une a expressividade de SQL com a eficiencia vetorizada de um DataFrame colunar moderno.

Na camada de atributos, o sistema suporta:

- abundancias brutas;
- TSS (*Total Sum Scaling*);

- CLR (*Centered Log-Ratio*);
- filtragem por prevalencia;
- reducao de dimensionalidade e diversidade.

Na camada de modelagem, o projeto ja implementa classes dedicadas para:

- `MicrobiomeClassifier`, com suporte a Random Forest, SVM e XGBoost;
- `MicrobiomeRegressor`;
- `MicrobiomeClusterer`;
- `HyperparameterTuner`;
- `SHAPAnalyzer`.

Essa organizacao fortalece a ideia de biblioteca. Em vez de o usuario escrever um notebook monolitico, ele passa a montar o fluxo com componentes especializados, todos consumindo um mesmo formato de matriz gerado a partir do banco local.

3.6. Interface local, servidor HTTP e relatorios

Embora o foco do periodo tenha migrado para a consolidacao da biblioteca, a camada de interface local permaneceu como parte importante da arquitetura. O comando de entrada `flora` e definido em `pyproject.toml` por meio do *entry point* `flora = flora.ui.cli:main`. A CLI atual ja expoe comandos para:

- iniciar a interface web local com `flora ui`;
- executar uma pipeline configurada com `flora run`;
- baixar datasets publicos com `flora download mgnify`, `sra`, `emp` e `synthetic`.

Na camada HTTP, `src/flora/ui/server.py` organiza o despacho das rotas e integra o estado da interface por meio de `FLORAState`. O frontend compilado da SPA Angular e servido a partir de `src/flora/ui/static/app/`, mantendo a operacao local sem necessidade de infraestrutura web externa. Isso reforca o conceito original do projeto: uma plataforma *friendly*, local e aberta.

Os relatarios sao produzidos por `FLORAReport`, em `src/flora/reports/html_report.py`. Esse componente monta um HTML autocontido com metricas, tabelas e graficos Plotly. Assim, o produto final da analise nao e apenas um banco ou um notebook, mas um artefato compartilhavel e auditavel.

3.7. Sequencia operacional de uma execucao completa

Do ponto de vista de engenharia de software, uma das contribuicoes mais relevantes do FLORA e tornar explicita a sequencia operacional de uma corrida analitica. Em um cenario tradicional, essas etapas ficam espalhadas em scripts, notebooks e chamadas manuais. No FLORA, a execucao segue um encadeamento previsivel, iniciado pelo usuario via CLI ou interface local.

```
sequenceDiagram
    actor U as Usuario
    participant CLI as flora CLI
    participant P as FLORAPipeline
    participant DB as FloraDB / DuckDB
    participant ING as db.ingestion
    participant ML as Modulos ML
    participant REP as FLORAReport

    U->>CLI: flora run config.yaml
    CLI->>P: inicializa pipeline
    P->>DB: connect() + initialize_schema() + create_views()
    CLI->>P: ingest_metadata()
    P->>ING: ingest_metadata()
    ING->>DB: load_dataframe(samples, sample_metadata)
    CLI->>P: ingest_asv_table()
    P->>ING: ingest_tsv_asv() ou ingest_biom()
    ING->>DB: load_dataframe(asv)
    CLI->>P: ingest_taxonomy()
    P->>ING: ingest_taxonomy()
    ING->>DB: load_dataframe(taxonomy)
    CLI->>P: compute_diversity()
    P->>DB: pivot_asv()
    P->>DB: load_dataframe(diversity_alpha)
    CLI->>P: get_feature_matrix(normalize="clr")
    P->>DB: pivot_asv(normalize="clr")
    CLI->>ML: fit(train_df, test_df)
    ML-->>CLI: metricas + importancias + previsoes
    CLI->>REP: add_metrics() / add_plot() / save()
    REP-->>U: report.html
```

Listing 5: Diagrama Mermaid de sequencia para uma corrida completa do FLORA.

O diagrama **??** mostra que o FLORA nao e apenas uma colecao de utilitarios: ele

define uma linha de montagem analitica que pode ser auditada, repetida e documentada. Esse nivel de explicitacao e particularmente importante em software cientifico, onde reproducibilidade tem valor metodologico.

3.8. Mapa de modulos do software

O repositorio atual permite descrever o sistema em camadas funcionais. A Tabela **??** resume os modulos centrais e seus papeis.

Table 2: Mapa resumido dos modulos de software do FLORA.

Modulo	**Arquivos principais**	**Responsabilidade**
`config`	`settings.py`	Define configuracao tipada por Pydantic para banco, pipeline, ML, logging e armazenamento.
`core`	`exceptions.py`, `logging.py`	Centraliza excecoes de dominio e configuracao de logging.
`db`	`connection.py`, `schema.py`, `ingestion.py`	Implementa banco DuckDB, esquema, ingestao, *views* e consultas analiticas.
`io`	`downloaders.py`, `validators.py`	Conecta o sistema a fontes externas e faz validacoes de entrada.
`diversity`	`alpha.py`, `beta.py`	Calcula diversidade alfa e beta.
`feature_engineering`	`normalization.py`, `selection.py`, `reduction.py`, `encoding.py`	Implementa normalizacao, selecao de atributos, codificacao e reducao de dimensionalidade.
`ml`	`classification/`, `regression/`, `clustering/`, `evaluation/`, `optimization/`, `explainability/`	Reune algoritmos de modelagem, metricas, tuning e interpretabilidade.
`reports`	`html_report.py`	Gera relatarios HTML autocontidos.
`viz`	`taxonomy_plots.py`, `diversity_plots.py`, `ml_plots.py`	Produz artefatos visuais para diversidade, taxonomia e desempenho de modelos.
`pipelines`	`main_pipeline.py`	Orquestra o fluxo ponta a ponta.

Modulo	Arquivos principais	Responsabilidade
ui	cli.py, server.py, frontend.py, handlers/	Exponhe a biblioteca como CLI, API HTTP local e frontend web.

Essa modularizacao facilita manutencao e extensao. Por exemplo, novos conectores podem ser adicionados em `io`, sem alterar a camada de ML; novas estrategias de normalizacao podem ser adicionadas em `feature_engineering`, sem modificar o servidor HTTP.

```
flowchart TB
    A[Presentation Layer\ncli / server / handlers / SPA] --> B[Application
    Layer\nFLORAPipeline]
    B --> C[Analytics Layer\nFloraDB / SQL / Views]
    B --> D[Domain Services\ndiversity / feature_engineering / ml / reports /
    viz]
    B --> E[External Adapters\ndownloaders / filesystem / config]
    C --> F[(DuckDB)]
    E --> G[APIs externas e arquivos locais]
```

Listing 6: Diagrama Mermaid das camadas de software do FLORA.

3.9. Contratos da interface CLI e da API local

Outro ponto importante para documentar o software e o contrato externo que ele expoe ao usuario. O FLORA fornece dois contratos principais: a CLI e a API HTTP local servida pela interface web.

3.9.1. CLI

A CLI, definida em `src/flora/ui/cli.py`, oferece atualmente tres familias de comandos:

- `flora ui`: sobe o servidor local e a interface web;
- `flora run`: executa a pipeline completa a partir de um arquivo de configuracao;
- `flora download`: baixa datasets publicos, com subcomandos para `mgnify`, `sra`, `emp` e `synthetic`.

Essa escolha de design e relevante porque evita que o usuario precise conhecer a estrutura interna do codigo. O software se torna consumivel como ferramenta, e nao apenas como repositório.

3.9.2. API HTTP local

Na camada HTTP, o servidor organiza rotas GET e POST para ingestao, visualizacao, relatorios e estado da aplicacao. A Tabela **??** resume os principais endpoints definidos atualmente.

Table 3: Principais endpoints da API local do FLORA.

Metodo	Endpoint	Funcao
GET	`/api/status`	Retorna estado do sistema, workdir, modo do frontend e informacao resumida das tabelas.
GET	`/api/logs`	Retorna buffer de logs para inspeção pela interface.
GET	`/api/feature_matrix`	Exponhe a matriz de atributos gerada pelo backend.
GET	`/api/viz/taxonomy`	Agrega abundancias por nivel taxonomico.
GET	`/api/viz/pcoa`	Retorna dados para PCoA e analise beta.
GET	`/api/viz/alpha`	Exponhe metricas de diversidade alfa por grupo.
GET	`/api/viz/taxonomy_heatmap`	Retorna estrutura para heatmap taxonomico.
GET	`/api/viz/beta_heatmap`	Retorna matriz de distancias beta.
GET	`/api/viz/krona`	Prepara dados hierarquicos para visualizacao estilo Krona.
GET	`/api/viz/tree`	Exponhe estrutura de arvore taxonomica.
GET	`/api/viz/network`	Exponhe relacoes para grafos de coocorrencia ou estrutura taxonomica.
POST	`/api/ingest/metadata`	Ingestao de metadados.
POST	`/api/ingest/asv`	Ingestao de tabela ASV.
POST	`/api/ingest/taxonomy`	Ingestao de taxonomia.
POST	`/api/diversity`	Dispara calculo de diversidade.
POST	`/api/ml/classify`	Dispara classificacao supervisionada.

Metodo	Endpoint	Funcao
POST	`/api/ml/cluster`	Dispara clustering.
POST	`/api/download/mgnify`	Aciona downloader de MGnify.
POST	`/api/download/sra`	Aciona downloader de SRA.
POST	`/api/download/emp`	Aciona downloader de EMP.
POST	`/api/download/synthetic`	Gera dados sinteticos.
POST	`/api/report`	Gera relatorio analitico.

Esse conjunto de endpoints e suficiente para sustentar uma interface web local desacoplada do nucleo cientifico. Em outras palavras, a UI consome contratos JSON, enquanto a logica de negocio permanece no backend Python.

```
sequenceDiagram
    actor U as Usuario
    participant FE as Frontend Angular
    participant API as servidor HTTP FLORA
    participant H as handlers/*
    participant ST as FLORAState
    participant P as FLORAPipeline
    participant DB as DuckDB

    U->>FE: abre interface local
    FE->>API: GET /api/status
    API->>H: status.get_status(...)
    H->>ST: consulta estado atual
    ST->>P: acesso ao pipeline
    P->>DB: table_info()
    DB-->>P: resumo das tabelas
    P-->>H: payload JSON
    H-->>API: resposta
    API-->>FE: status do sistema
    U->>FE: solicita grafico ou ingestao
    FE->>API: GET/POST /api/...
    API->>H: handler especifico
    H->>P: executa caso de uso
    P->>DB: consulta ou escrita
    DB-->>P: resultado
    P-->>H: payload
    H-->>API: JSON
    API-->>FE: atualiza tela
```

Listing 7: Diagrama Mermaid da interacao entre frontend SPA e backend local.

3.10. Empacotamento da biblioteca e estrategia de publicacao

Um aspecto central para a maturidade do projeto e a preparacao para publicacao como biblioteca reutilizavel. O arquivo `pyproject.toml` ja define:

- nome do projeto: `flora-bio`;
- versao atual: `0.1.0`;
- *build backend*: `hatchling.build`;
- dependencias obrigatorias e extras opcionais;
- *entry point* de CLI;
- metadados de homepage, documentacao, repositorio e issues.

Isso significa que o repositorio ja nao esta organizado apenas para uso local do desenvolvedor. Ele foi estruturado para gerar uma distribuicao wheel/sdist, instalar via `pip`, manter extras para ambientes opcionais e, posteriormente, ser publicado em um indice de pacotes como PyPI. Em termos de engenharia de software, isso muda o status do projeto: o FLORA passa a ser pensado como produto distribuivel.

```
flowchart TD
    A[Codigo fonte em src/flora] --> B[pyproject.toml]
    B --> C[Hatchling build backend]
    C --> D[Wheel / sdist]
    D --> E[Instalacao local com pip install -e .]
    D --> F[Publicacao futura no PyPI]
    A --> G[Testes com pytest]
    A --> H[Documentacao tecnica]
    G --> C
    H --> F
    F --> I[Usuario final instala flora-bio]
    I --> J[CLI flora]
    I --> K[Uso programatico via import flora]
```

Listing 8: Diagrama Mermaid do ciclo de empacotamento e publicacao da biblioteca.

No contexto do relatorio parcial, ainda e mais adequado afirmar que a biblioteca esta **preparada para publicacao** do que afirmar que ja esta publicada. Mesmo assim, a infraestrutura necessaria para esse passo ja existe e foi efetivamente implementada.

3.11. Nota sobre os diagramas Mermaid

Os diagramas Mermaid deste documento foram incluidos em formato textual diretamente no fonte LaTeX, o que facilita sua manutencao junto ao codigo e a reutilizacao em README, MkDocs, documentacao web e apresentacoes. Em uma etapa posterior, esses mesmos blocos podem ser renderizados como figuras SVG ou PNG para incorporacao visual direta ao PDF final.

3.12. Qualidade de software, testes e observabilidade

Uma biblioteca cientifica nao deve ser avaliada apenas pela existencia de algoritmos. Ela precisa oferecer previsibilidade operacional. Nesse aspecto, o FLORA avancou em quatro direcoes.

Primeiro, o projeto possui uma estrutura formal de testes em `tests/`, incluindo casos unitarios e de integracao para componentes como frontend, servidor HTTP, camada de HTTP utilitaria e bootstrap da UI. Segundo, a camada de excecoes em `src/flora/core/exceptio` padroniza erros de pipeline, validacao, banco, ingestao e ML, reduzindo falhas opacas. Terceiro, a tabela `pipeline_log` e o buffer de logs do servidor adicionam rastreabilidade operacional. Quarto, o proprio `pyproject.toml` define execucao padronizada de testes com `pytest` e cobertura.

Do ponto de vista de software cientifico, essa preocupacao e essencial. Sem logs, testes e contratos claros, o pipeline pode ate funcionar em um computador de desenvolvimento, mas nao se sustenta como ferramenta reutilizavel. Ao incluir auditoria de execucao e testes automatizados, o FLORA se aproxima de uma base de software mais robusta para pesquisa.

```
flowchart LR
    A[Codigo fonte] --> B[pytest]
    A --> C[ruff / mypy]
    A --> D[logging estruturado]
    D --> E[pipeline_log]
    D --> F[/api/logs]
    B --> G[Confiabilidade]
    C --> G
    E --> H[Rastreabilidade]
```

```
F --> H
G --> I[Release mais segura]
H --> I
```

Listing 9: Diagrama Mermaid da camada de qualidade de software do FLORA.

3.13. Implicacoes cientificas da arquitetura de software

Do ponto de vista academico, detalhar a arquitetura do software nao e um desvio do problema biologico; e parte do proprio resultado cientifico. O FLORA propoe que uma analise de microbioma possa ser reproduzida em uma cadeia fechada de transformacoes observaveis, indo do download do dado bruto ao relatorio final. Isso tem pelo menos quatro implicacoes.

- melhora reproducibilidade, porque reduz etapas manuais e estados ocultos;
- melhora auditabilidade, porque os dados e resultados ficam concentrados em um banco local e em logs de pipeline;
- melhora extensibilidade, porque novos modelos e novos conectores podem ser acoplados sem reescrever o sistema inteiro;
- melhora transferibilidade, porque o software fica apto a ser instalado por terceiros como biblioteca Python.

Em um projeto PIBIC com foco em software cientifico, essa transicao de *script de pesquisa* para *biblioteca reproduzivel* constitui, por si so, um resultado relevante.

4. Resultados Parciais e Discussao

No periodo avaliado, houve uma redefinicao arquitetural importante. A proposta inicial dava maior peso a uma interface grafica, mas a experiencia de desenvolvimento mostrou que a sustentabilidade do projeto dependia da consolidacao previa de um nucleo cientifico robusto. A partir disso, o FLORA passou a ser organizado como biblioteca modular, sobre a qual interfaces, relatorios e automatizacoes podem ser construidos com menor risco tecnico.

Os principais avancos consolidados foram:

- **camada DuckDB estabilizada:** esquema relacional local com 8 tabelas e 6 indices, mais *views* analiticas para TSS, CLR, diversidade e taxonomia;

- **conectores de dados publicos:** suporte a MGnify, EMP e NCBI SRA com escrita local de manifestos e metadados;
- **pipeline composicional:** geracao automatica de matrizes normalizadas por *raw*, TSS e CLR com filtragem por prevalencia;
- **camada de ML funcional:** classificacao, regressao, clustering, tuning e interpretabilidade;
- **relatorios e interface local:** producao de HTML autocontido, API HTTP local e frontend SPA;
- **preparacao para distribuicao:** estrutura de pacote compatível com `pip`, Hatchling e futura publicacao.

4.1. Sintese do que foi efetivamente desenvolvido

Considerando o estado atual do repositorio, o FLORA ja entrega um conjunto coerente de componentes cientificos e operacionais:

- ingestao de `metadata.tsv`, `taxonomy.tsv`, TSVs de ASV em formato largo ou longo e arquivos BIOM;
- banco analitico local em DuckDB com suporte a Parquet, Arrow e DataFrames Polars;
- matrizes para ML produzidas por SQL e exportadas em formato tabular;
- diversidade alfa e beta, incluindo Shannon, Chao1, Simpson, Bray-Curtis e PCoA;
- modelos supervisionados e nao supervisionados para exploracao dos dados;
- relatorio HTML autocontido com metricas e graficos;
- CLI unica para download, execucao da pipeline e interface local.

Esse conjunto mostra que o projeto ja ultrapassou a fase de prototipo conceitual. O nucleo do trabalho hoje nao e apenas construir interface, mas consolidar uma infraestrutura local reproduzivel para microbioma, com foco simultaneo em ingestao, modelagem, persistencia e interpretacao.

4.2. Experimento com Earth Microbiome Project

Foi executada uma pipeline completa com dados locais do Earth Microbiome Project. A Tabela **??** resume os principais indicadores obtidos.

Table 4: Resumo quantitativo da pipeline EMP no FLORA.

Indicador	**Valor**
Amostras ingeridas	89
Amostras usadas no modelo	85
Observacoes ASV carregadas	28.857
Taxonomias carregadas	197.711
Features apos filtro de prevalencia	1.183
Acuracia no conjunto de teste	0,8636
F1-macro no conjunto de teste	0,6381
ROC-AUC	0,9510
Shannon medio	3,5223
Chao1 medio	354,84

Os resultados sugerem que a estrutura do dado possui sinal suficiente para discriminacao entre categorias ambientais selecionadas. O ROC-AUC elevado indica boa capacidade discriminativa global. O F1-macro mais moderado, por sua vez, sugere que o desbalanceamento entre classes ainda afeta a distribuicao dos erros. Esse padrao e compativel com o tipo de tarefa e reforca a necessidade de usar metricas para alem da acuracia agregada.

4.3. Comparacao metodologica

Um avanco importante do periodo foi a introducao de comparacao entre configuracoes metodologicas distintas. No benchmark atual, `CLR + XGBoost` apresentou o melhor equilibrio entre acuracia, *balanced accuracy*, F1-macro e ROC-AUC. Em paralelo, `TSS + Random Forest` tambem apresentou desempenho elevado, enquanto o *baseline* majoritario confirmou que a tarefa nao pode ser analisada apenas pela frequencia da classe dominante.

Table 5: Comparacao resumida entre configuracoes metodologicas no benchmark EMP.

Normalizacao	Modelo	Accuracy	Bal. Acc.	F1-macro	ROC-AUC
Raw	Majority	0,7529	0,3333	0,2863	0,5000
Raw	XGBoost	0,9294	0,8252	0,8204	0,9946
TSS	Random Forest	0,9471	0,8838	0,8801	0,9941
TSS	XGBoost	0,9471	0,8782	0,8757	0,9952
CLR	Random Forest	0,9471	0,8615	0,8721	0,9924
CLR	XGBoost	0,9529	0,8919	0,8903	0,9936
CLR	SVM	0,9294	0,8085	0,8072	0,9874

Os dados da Tabela **??** mostram que o ganho metodologico da normalizacao nao deve ser lido apenas pela acuracia. Em problemas desbalanceados, metricas como F1-macro e *balanced accuracy* descrevem melhor se o modelo esta tratando adequadamente as classes minoritarias. Nesse sentido, a configuracao `CLR + XGBoost` foi a mais favoravel entre as avaliadas.

4.4. Relacao entre ML e interpretacao biologica

Outro avanco importante foi a reconexao entre atributos discriminantes e grupos taxonomicos plausiveis. Entre os filos mais frequentes nas top features apareceram Proteobacteria, Planctomycetes, Firmicutes e Bacteroidetes. Isso fortalece a interpretacao do modelo, pois o resultado deixa de ser apenas numerico e passa a ser discutivel em termos de assinaturas microbianas e contexto ecologico.

Table 6: Sinal taxonomico agregado das features mais discriminantes.

Phylum	N features	Score total
Proteobacteria	11	0,0571
Planctomycetes	2	0,0110
Firmicutes	1	0,0088
Bacteroidetes	1	0,0058
Chloroflexi	1	0,0047
Acidobacteria	1	0,0043

Essa ponte entre atributo computacional e filo taxonomico e central para a argumentacao cientifica do trabalho. Em vez de produzir apenas um classificador com boa metrica, o FLORA permite perguntar quais grupos microbianos sustentam a separacao entre ambientes e se esses grupos sao ecologicamente plausiveis.

4.5. Interpretacao dos graficos

As figuras geradas cumprem papeis complementares no relatorio:

- **PCoA Bray-Curtis:** mostra estruturacao global e separacao composicional entre grupos ambientais;
- **Shannon e Chao1:** quantificam diversidade local sob perspectivas complementares de uniformidade e riqueza;
- **barplot e heatmap taxonomico:** conectam separacao estatistica a assinaturas biologicas observaveis;
- **ROC e matriz de confusao:** permitem auditar desempenho do classificador;
- **importancia de features e SHAP:** indicam quais ASVs sustentam as decisoes do modelo.

4.6. Robustez experimental

As analises de sensibilidade adicionadas ao benchmark reforcam que as decisoes de preprocessamento alteram materialmente os resultados. Em relacao a profundidade de rarefacao, o melhor compromisso observado ocorreu em `sampling depth = 5000`, com *accuracy* de 0,9803, *balanced accuracy* de 0,9883 e F1-macro de 0,9693. Em profundidades mais altas, a perda de amostras reduziu parte da robustez. De modo semelhante, o filtro de prevalencia mostrou que valores intermediarios entre 0,03 e 0,10 preservam desempenho competitivo com forte reducao de dimensionalidade, enquanto filtros muito agressivos removem sinal biologico util.

Table 7: Resumo das analises de robustez para a configuracao `CLR + XGBoost`.

Cenario	Accuracy	Bal. Acc.	F1-macro	N features
Rarefacao 3000	0,9375	0,7057	0,6931	995
Rarefacao 5000	0,9803	0,9883	0,9693	1193
Rarefacao 8000	0,9857	0,9648	0,9544	1048
Rarefacao 10000	0,9609	0,8667	0,8228	1005
Prevalencia 0,03	0,9416	0,8446	0,8475	2616
Prevalencia 0,10	0,9356	0,8420	0,8441	532

4.7. Figuras sugeridas para o relatorio

As figuras da pipeline principal ja existem como artefatos prontos em `docs/assets/emp`. Para facilitar a finalizacao do relatorio parcial, foram incluidos abaixo espacos reservados que incorporam automaticamente a imagem, quando disponivel, ou mantem um placeholder compativel com compilacao.

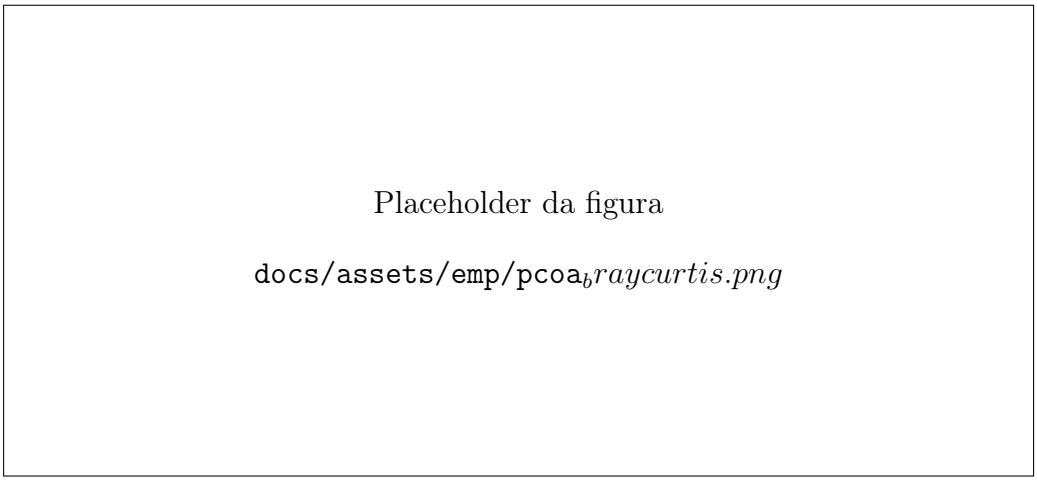


Figure 1: PCoA Bray-Curtis mostrando separacao global entre os grupos ambientais utilizados no experimento EMP.

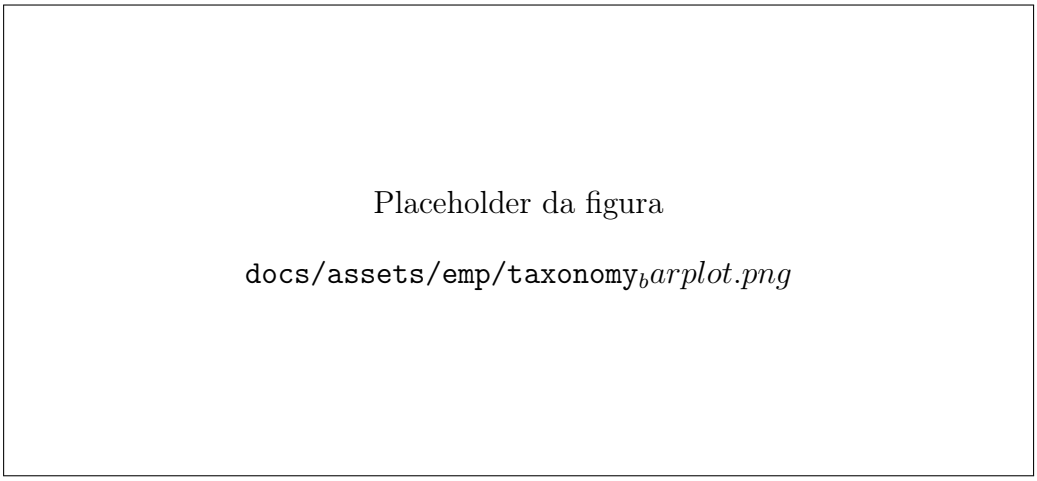


Figure 2: Composicao taxonomica media por classe ambiental, usada para relacionar a separacao estatistica a assinaturas biologicas observaveis.

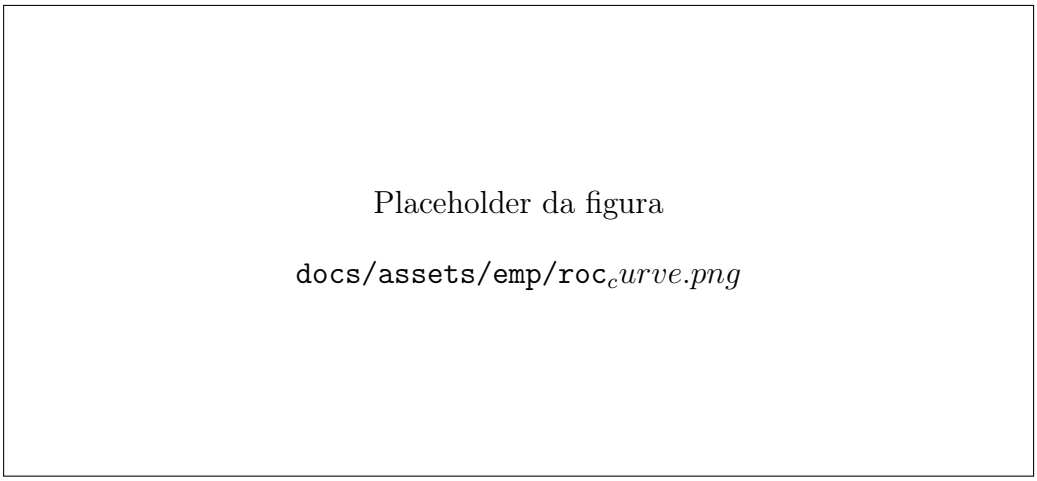


Figure 3: Curva ROC do classificador supervisionado no experimento EMP.

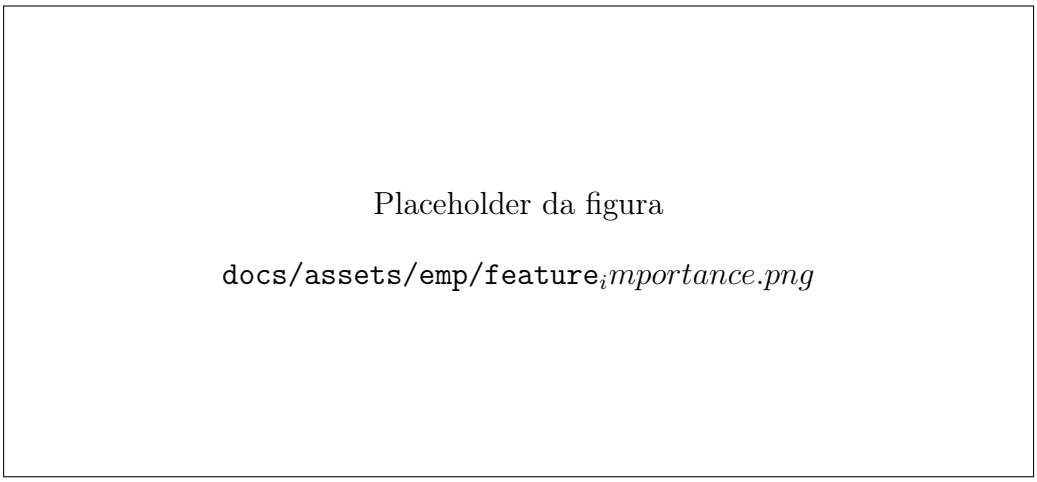


Figure 4: Importancia das features mais discriminantes no modelo supervisionado.

4.8. Limitacoes

Apesar dos avancos, algumas limitacoes precisam ser reconhecidas explicitamente:

- a secao biologica ainda pode detalhar com maior rigor primers, regiao-alvo e parametros de *denoising*;
- a validacao principal ainda esta concentrada no EMP e deve ser ampliada para mais datasets reais;
- a camada de SHAP ainda demanda estabilizacao ambiental em alguns cenarios;
- a documentacao final para publicacao em PyPI, Read the Docs e artigo cientifico ainda precisa ser integrada em uma versao estavel.

5. Maturidade do Produto de Software

Do ponto de vista de engenharia, o FLORA ja apresenta caracteristicas tipicas de um produto cientifico serio:

- empacotamento padronizado via `pyproject.toml`;
- separacao razoavel entre nucleo analitico e camada de interface;
- testes automatizados em `tests/`;
- estrutura de documentacao em `docs/`;
- suporte a uso por CLI, API local e frontend web;
- banco analitico local capaz de sustentar reproducao do estado do experimento.

Em termos de ciclo de vida, o projeto se encontra em uma fase intermediaria: ja existe um artefato utilizavel por desenvolvedores e pesquisadores com familiaridade tecnica, mas ainda resta consolidar interfaces, validacao externa e documentacao de usuario para ampliar a adocao.

6. Cronograma e Proximos Passos

Para o proximo periodo, o projeto deve concentrar esforcos em quatro frentes:

- ampliar a validacao em novos datasets publicos alem do EMP;
- fortalecer a documentacao voltada a instalacao, uso e publicacao do pacote;
- estabilizar a camada de interpretabilidade e refinar a integracao frontend-backend;
- elevar a robustez estatistica e a cobertura de testes para o ciclo de release.

6.1. Plano de acao imediato

Semana 1: consolidar a versao textual final da arquitetura, banco local e estrategia de publicacao no relatorio e na documentacao tecnica.

Semana 2: ampliar benchmark em novos conjuntos publicos e consolidar tabelas comparativas centrais.

Semana 3: preparar versao candidata de distribuicao da biblioteca, revisar dependencias opcionais e fortalecer a experiencia de instalacao.

7. Conclusao Parcial

O desenvolvimento do FLORA no periodo analisado resultou em avancos tecnicos consistentes e alinhados aos objetivos de uma iniciacao cientifica em software cientifico. A mudanca de foco, da interface para a consolidacao da biblioteca, mostrou-se correta do ponto de vista metodologico. O projeto ja oferece aquisicao de dados publicos, ingestao de dados reais, armazenamento analitico robusto, transformacoes composicionais, modelagem supervisionada, visualizacao, interface local e relatorios reproduziveis.

Como resultado, o FLORA se posiciona nao apenas como um prototipo de interface, mas como uma infraestrutura computacional em evolucao para analise de microbiomas. A documentacao apresentada neste relatorio, incluindo os diagramas Mermaid de arquitetura, classes, fluxo de dados externos, banco DuckDB e empacotamento, mostra que a biblioteca ja possui um desenho tecnico coerente o suficiente para sustentar crescimento, manutencao e futura publicacao.

O passo seguinte nao e mais apenas programar novas funcionalidades. E demonstrar, com validacao experimental mais ampla e documentacao madura, que o software construido pode ser reutilizado por outros pesquisadores como ferramenta cientifica confiavel, aberta e local.

Referencias Bibliograficas

[1] BOLYEN, E. et al. Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. **Nature Biotechnology**, v. 37, p. 852–857, 2019.

[2] CALLAHAN, B. J. et al. DADA2: High-resolution sample inference from Illumina amplicon data. **Nature Methods**, v. 13, p. 581–583, 2016.

[3] GLOOR, G. B. et al. Microbiome datasets are compositional: and this is not optional. **Frontiers in Microbiology**, v. 8, 2224, 2017.

[4] RAASVELDT, M.; MUEHLEISEN, H. DuckDB: an embeddable analytical database. **Proceedings of SIGMOD 2019**, p. 1981–1984, 2019.

[5] LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. **Advances in Neural Information Processing Systems**, v. 30, 2017.

Joao Pedro Pereira Santiago
Bolsista de Iniciacao Cientifica

Prof. Dr. Diego Lisboa Rios
Orientador do Projeto